GENERACIÓN DE CÓDIGO CONCURRENTE MEDIANTE VIBECODING: ANÁLISIS CRÍTICO Y MÉTRICAS DE UN SIMULADOR DE CENTRO LOGÍSTICO EN PYTHON

Integrantes:

Santiago ALVAREZ [(1)] salvarez927@alumno.unlam.edu.ar

Victoria BLANCO [(2)] viblanco@alumno.unlam.edu.ar

Santiago BRIASCO CALVO [(3)] sbriascocalvo@alumno.unlam.edu.ar

Octavio Dion GEMELLARO [(4)] ogemellaro@alumno.unlam.edu.ar

Micaela Abril MARRONE [(5)] mmarrone584@alumno.unlam.edu.ar

Tomas Agustín MENENDEZ [(6)] tmenendez@alumno.unlam.edu.ar

Asignatura:

(01-3900) PROGRAMACIÓN CONCURRENTE

Universidad Nacional de La Matanza — Departamento de Ingeniería e Investigaciones Tecnológicas

Fecha - Versión:

15/10/2026 – Versión 1.00

Actividad Práctica Obligatoria N.° 1 — 2.° cuatrimestre 2026

Resumen:

Este trabajo analiza la generación de un programa concurrente de complejidad no trivial mediante herramientas de VibeCoding. A partir de un prompt técnico diseñado en grupo con meta-prompting, encadenamiento de razonamiento y asignación de rol, se generó en Python un simulador de centro logístico que combina los problemas productor-consumidor y lectores-escritores, con cierre ordenado en dos fases, pausas de mantenimiento aleatorias y persistencia atómica en JSON. El código se construyó de forma incremental con agentes de IA en ramas por funcionalidad y se evaluó con las rúbricas de la cátedra, 37 pruebas automatizadas y mediciones de CPU, memoria, throughput y tokens en cuatro escenarios de hasta 104 hilos. No se registraron pérdidas ni duplicaciones de paquetes y el uso medio de CPU no superó el 3,2 %, lo que confirma la ausencia de espera activa. El análisis crítico identificó defectos que la IA no anticipó —incompatibilidades de E/S en Windows y un bloqueo ante una segunda—, lo que muestra que la supervisión humana y las pruebas siguen siendo indispensables.

Palabras Clave: *programación concurrente, generación de código con IA, productor-consumidor, lectores-escritores, métricas de software*

I. CONTEXTO

El trabajo se inscribe en el área de Ingeniería de Software, en la intersección entre la programación concurrente y la generación automática de código mediante modelos de lenguaje de gran escala (LLM). Corresponde a la Actividad Práctica Obligatoria N.° 1 de la asignatura Programación Concurrente de la carrera de Ingeniería en Informática de la Universidad Nacional de La Matanza (UNLaM), dictada en el segundo cuatrimestre de 2026. El proyecto no cuenta con financiamiento externo.

La consigna pide diseñar en grupo un prompt técnico, generar con VibeCoding un programa concurrente en Python, Java o Kotlin, analizar críticamente el resultado y evaluarlo con métricas de cumplimiento funcional, correctitud concurrente, uso de CPU, uso de memoria y tokens consumidos. El código, las pruebas y el historial de desarrollo están disponibles en el repositorio del grupo: `https://github.com/UNLAM-PROG-C/2026-PROGC-Q2-M4/tree/develop/TP%20Metrica`.

II. INTRODUCCIÓN

La programación concurrente estructura un programa como un conjunto de actividades que progresan de manera intercalada y comparten recursos. Sus fundamentos se remontan a los procesos secuenciales cooperantes y los semáforos de Dijkstra [1], y a los monitores de Hoare [2], que encapsulan el estado compartido junto con su sincronización. Dos problemas clásicos sirven aún como banco de pruebas: el productor-consumidor, que coordina procesos que generan y consumen datos a través de un búfer acotado [1], y el de lectores-escritores, que permite lecturas concurrentes y exige exclusión mutua para las escrituras [3]. A ellos se suma el análisis de interbloqueos y sus condiciones necesarias [4].

Los errores de concurrencia siguen siendo difíciles de detectar. En un estudio de 105 errores reales en aplicaciones de código abierto, Lu et al. [5] observaron que la mayoría de los errores sin interbloqueo eran violaciones de atomicidad o de orden, y que muchos dependían de intercalados poco frecuentes que las pruebas convencionales no ejercitan. Por eso las técnicas modernas privilegian abstracciones de alto nivel —colas bloqueantes, objetos inmutables, señales de terminación— frente a la manipulación directa de candados [6].

En paralelo, los LLM se convirtieron en asistentes de programación de uso masivo. Una revisión sistemática de 395 trabajos muestra que la generación de código es la tarea de ingeniería de software más estudiada con LLM [7]. Ziegler et al. [8] midieron mejoras de productividad percibida con GitHub Copilot. Pearce et al. [9], en cambio, encontraron que alrededor del 40 % de los programas generados en escenarios de riesgo contenían vulnerabilidades. Nichols et al. [10] evaluaron LLM en la generación de código paralelo y hallaron que su desempeño en estas tareas es notablemente inferior al de la programación secuencial. El término *vibe coding*, popularizado en 2025 [11], describe un estilo en el que el desarrollador guía al modelo en lenguaje natural y evalúa el resultado por su comportamiento más que por la lectura línea a línea del código [12].

Evaluar ese código exige métricas con fundamento. La teoría de la medición en software establece que toda métrica debe definir qué atributo mide, con qué instrumento y en qué escala [13]. El análisis de desempeño, por su parte,

requiere fijar la carga de trabajo, repetir las mediciones y reportar su variabilidad [14], [15]. Este trabajo aplica esos criterios a un programa concurrente generado con IA, con un doble objetivo: verificar si el código cumple su función de manera correcta y eficiente, e identificar qué tipo de defectos introduce la generación asistida y cuáles requieren supervisión humana.

III. MÉTODOS

A. Diseño del prompt técnico

El problema se eligió con una primera consulta exploratoria a un LLM, a la que se le adjuntaron las rúbricas de la cátedra. De las propuestas obtenidas, el grupo eligió un **simulador de centro logístico**, porque integra en un mismo escenario el patrón productor-consumidor (camiones que descargan paquetes en un muelle) y el de lectores-escritores (operarios que actualizan un inventario mientras otros hilos lo consultan).

Para el prompt de inicialización se descartó el *few-shot prompting*, pensado para enseñar patrones de entrada y salida, y se combinaron tres técnicas: **asignación de rol** (el modelo actúa como arquitecto de software especializado en concurrencia), **meta-prompting** (el prompt describe la estructura de la solución antes que su contenido) y **encadenamiento de razonamiento** (un plan explícito de cuatro pasos). Las restricciones del prompt se derivaron directamente de los criterios de las rúbricas: sin espera activa, colas de la biblioteca estándar, exclusión mutua con contextos `with`, *timeouts* en toda operación bloqueante, cierre ordenado con `threading.Event` y manejo global de excepciones por hilo. A continuación se transcribe el prompt utilizado.

```
Actúa como un Arquitecto de Software Senior especializado en
programación concurrente en Python. Tu objetivo es inicializar la
estructura base y el código fundacional para un "Simulador de Centro
Logístico e Inventario".

CONTEXTO DEL SISTEMA:
- Productores (Camiones que insertan lotes de paquetes en una cola).
- Consumidores/Escritores (Operarios que extraen paquetes y actualizan
  un inventario central).
- Lectores (Monitor/Dashboard que lee el inventario y la cola).

RESTRICCIONES ARQUITECTÓNICAS ESTRICTAS:
1. Sincronización: prohibido el busy waiting; usar queue.Queue.
2. Exclusión mutua: todo acceso al estado compartido protegido con
   with threading.Lock() o threading.RLock().
3. Tolerancia a fallos y deadlocks: timeouts en operaciones
   bloqueantes y graceful shutdown con threading.Event.
4. Calidad: type hints, docstrings (Google) y manejo global de
   excepciones en run() de cada hilo.

PLAN DE EJECUCIÓN (piensa paso a paso):
PASO 1: estructura de directorios (core, workers, ui, tests).
PASO 2: core/inventory.py con la clase Inventory y sus Locks.
PASO 3: workers/base.py, clase base abstracta sobre threading.Thread
        con threading.Event y bloque try/except general.
PASO 4: main.py con 2 camiones y 3 operarios, captura de
        KeyboardInterrupt y join() ordenado.
No implementes la UI todavía: prioriza bases de concurrencia robustas.
```

Sobre la base generada, el desarrollo continuó con cuatro prompts incrementales, uno por funcionalidad: panel de monitoreo (lectores), pausas aleatorias de mantenimiento (*chaos engineering* [16]), persistencia atómica en JSON y

configuración por línea de comandos. Cada prompt fijó la rama de trabajo, los archivos a crear, las restricciones de concurrencia, las pruebas requeridas y la convención de *commits* atómicos.

B. Modelos y herramientas de IA

La planificación (elección del problema, prompt inicial y plan de fases) se hizo con **Gemini 3.1 Pro** (Google) en modo conversacional. La generación de código se hizo con agentes ejecutados localmente sobre el repositorio y configurados con **gentle-ai**, una herramienta que agrega a agentes existentes memoria persistente, *skills* y un flujo de trabajo de desarrollo orgánico (ODD) con revisiones automáticas. Todas las fases de código se generaron con **Claude Code** usando el modelo **Claude Opus 5.5** (Anthropic). Así, el mismo agente y el mismo modelo se usaron en todo el proyecto, lo que hace comparables los consumos de cada fase. La Tabla I resume el proceso. Cada fase se desarrolló en una rama propia y se integró en `DesarrolloAPO` con *merge* sin avance rápido, de modo que el historial conserva la trazabilidad de cada funcionalidad.

TABLA I
FASES DE DESARROLLO ASISTIDO POR IA

Fase	Rama	Contenido generado	Commits
0	feature/concurrency-core	Inventario con Lock, Package inmutable, BaseWorker, Truck, Operator, main con cierre en dos fases	7
1	feature/ui-dashboard	Dashboard de terminal (hilo lector)	2
2	feature/chaos-monkey-interrupts	Supervisor con pausas aleatorias y prueba de estrés	4
3	feature/async-persistence	ExportWorker con escritura atómica, corrección para Windows	7
4	feature/cli-arguments	Argumentos CLI, correcciones del revisor, resumen final, README	11

Nota: se cuentan los commits de código de cada rama, sin los merges. Fuente: historial de Git de la rama DesarrolloAPO.

C. Arquitectura del código generado

El programa resultante tiene 962 líneas de código de producción en 12 módulos y 1069 líneas de pruebas en 11 archivos. La Fig. 1 muestra la arquitectura. Los hilos `Truck` (productores) descargan lotes de paquetes en un muelle `queue.Queue` de capacidad acotada. Los hilos `Operator` (consumidores y escritores) los extraen y los registran en `Inventory`, que protege su estado con un único `threading.Lock` y entrega a los lectores (`Dashboard` y `ExportWorker`) instantáneas inmutables. El hilo `Supervisor` activa y desactiva un `maintenance_event` que pausa a productores y consumidores. Todos los hilos heredan de `BaseWorker`, que aplica el patrón *Template Method*: el bucle, la verificación de terminación y el manejo global de excepciones están en la clase base, y cada subclase implementa una única unidad de trabajo acotada en `_step()`.

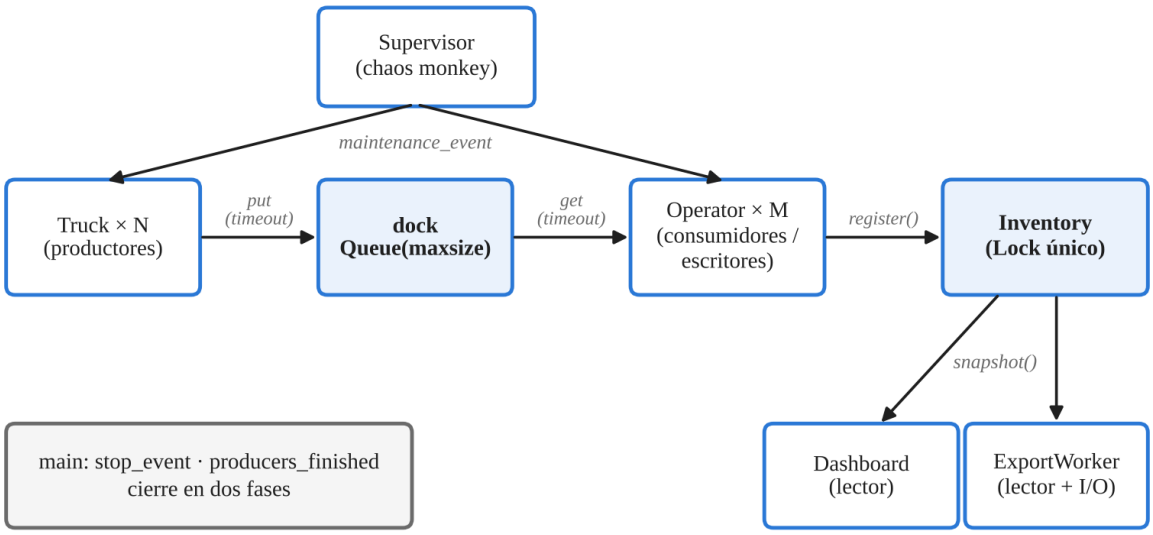


Fig. 1. Arquitectura del simulador. Las flechas indican el flujo de datos y de señales. Los recuadros sombreados son los recursos compartidos con sincronización interna.

El cierre se realiza en dos fases. Ante Ctrl+C, `stop_event` detiene a camiones, lectores y supervisor. Recién cuando todos se detuvieron, se activa `producers_finished`, y los operarios terminan solo cuando esa señal está activa y el muelle está vacío. Como después de esa señal nadie agrega paquetes, la condición de salida no tiene carrera y ningún paquete entregado se pierde.

D. Métricas e instrumentos de medición

Para cada métrica pedida se eligió el instrumento que mide el atributo de forma más directa y reproducible [13]. La Tabla II resume la justificación de cada elección y su fundamento teórico.

TABLA II
MÉTRICAS, INSTRUMENTOS Y JUSTIFICACIÓN

Métrica	Instrumento	Justificación y fundamento
Cumplimiento de funcionalidades	Rúbrica de funcionalidad, 37 pruebas unitarias (unittest) y ejecución manual	La rúbrica fija los criterios de evaluación. Las pruebas automatizadas hacen que la verificación sea repetible y objetiva.
Correctitud en concurrencia	Rúbrica de concurrencia, pruebas de estrés e invariante de conservación	Los errores de concurrencia dependen del intercalado [5]. Se verifica una invariante global (paquetes entregados = paquetes registrados, muelle vacío al cerrar) en ejecuciones repetidas con hasta 104 hilos.
Uso de CPU	psutil: cpu_percent cada 0,5 s y cpu_times al final	Mide el tiempo de CPU real del proceso, no una estimación. En un sistema cuyos hilos esperan la mayor parte del tiempo, un consumo alto delataría espera activa. Con el GIL de CPython, los hilos sirven para concurrencia, no para paralelismo [18].
Uso de memoria	psutil: RSS cada 0,5 s y corrida larga de 120 s	El RSS mide la memoria física efectivamente ocupada, incluidas las pilas de los hilos. Su evolución en el tiempo permite detectar fugas.
Tokens utilizados	Comando /cost de Claude Code (por sesión y fase)	Lo reporta el propio agente a partir de cada llamada a la API, con el detalle de entrada, salida y caché. Es un indicador del costo computacional y económico de la generación.

Las mediciones de CPU y memoria se hicieron con un script propio (`medir_metricas.py`, adjunto a la entrega) que ejecuta `main.py` como un proceso hijo, sin modificar el código. El script lo muestrea con psutil y lo detiene con SIGINT, que equivale a Ctrl+C. Se definieron cuatro escenarios: A, con la configuración por defecto (2 camiones y 3 operarios); B, con 10 y 10; C, de estrés, con 50 y 50; y D, igual a A pero sin el supervisor. Cada escenario se ejecutó **3**

veces durante 20 s, siguiendo la recomendación de repetir y promediar las mediciones [15], y se descartó el primer segundo de cada corrida como período de calentamiento. El entorno principal fue una PC con: Ubuntu sobre WSL2 en Windows, con 16 CPU lógicas (AMD Ryzen 7 5700, 8 núcleos), 16 GB de RAM y CPython 3.12.3. Para verificar que los resultados no dependen del equipo, la misma batería se replicó en un segundo entorno, un contenedor Linux con 2 vCPU Intel Xeon de 2,1 GHz, 8 GB de RAM y CPython 3.13. En ese entorno se hizo además una corrida de 120 s del escenario B para evaluar la estabilidad de la memoria.

IV. RESULTADOS Y OBJETIVOS

A. Cumplimiento de funcionalidades

Las 37 pruebas automatizadas pasan en tres ejecuciones consecutivas (19,3 s por ejecución). El programa arranca, muestra el panel en tiempo real, exporta el reporte cada 5 s y, al interrumpirse, imprime un resumen final que coincide exactamente con el JSON persistido. La Tabla III presenta la autoevaluación según la rúbrica de funcionalidad.

TABLA III
AUTOEVALUACIÓN SEGÚN LA RÚBRICA DE FUNCIONALIDAD

Criterio	Peso	Evidencia	Puntaje
Funciona sin errores	40	37/37 pruebas OK en 3 ejecuciones; 25/25 corridas medidas con código de salida 0	40
Cumplimiento del objetivo	40	Productor-consumidor y lectores-escritores integrados; cierre sin pérdida; persistencia; configuración por CLI	40
Diseño de la app	20	Panel de terminal limpio y resumen final; argumentos -h documentados. Interfaz textual simple, sin biblioteca gráfica	10
Total	100		90

Nota: puntajes propuestos por el grupo, sujetos a la evaluación de la cátedra.

B. Correctitud en concurrencia

En las 25 ejecuciones medidas en ambos entornos, la cantidad de paquetes registrados en el inventario coincidió exactamente con la suma de los paquetes entregados por los camiones, y el muelle quedó vacío al cerrar. Esto incluye el escenario de estrés con 104 hilos y la corrida larga, con 4403 paquetes y 24 pausas de mantenimiento. Es decir, no se observaron pérdidas, duplicaciones ni interbloqueos. La Tabla IV resume la autoevaluación según la rúbrica de concurrencia.

TABLA IV
AUTOEVALUACIÓN SEGÚN LA RÚBRICA DE CONCURRENCIA

Criterio	Peso	Mecanismo y evidencia	Puntaje
Sincronización	25	Queue bloqueante con timeout y Events para el ciclo de vida. La espera durante el mantenimiento consulta el estado cada 0,1 s, porque Event no permite esperar a que se desactive	20
Exclusión mutua	25	Un único Lock en Inventory; el lock no se mantiene durante la E/S de disco ni la de terminal	25
Condiciones de carrera	25	Package inmutable; instantáneas inmutables; invariante de conservación verificada en 25 corridas	24
Deadlocks	25	Timeouts en toda espera; un solo lock, sin anidamiento; cierre robusto ante una segunda interrupción (corregido en la fase 4)	23
Total	100		92

Nota: puntajes propuestos por el grupo, sujetos a la evaluación de la cátedra.

C. Uso de CPU y de memoria

La Tabla V y la Fig. 2 muestran los resultados de los cuatro escenarios en el entorno principal. El uso de CPU fue muy bajo en todos los casos: 0,17 % con la configuración por defecto y 3,20 % con 104 hilos, con desvíos de 0,08 puntos o menos entre repeticiones. Esto confirma que los hilos pasan casi todo el tiempo bloqueados en `Queue.get`, `Queue.put` o `Event.wait` y no en espera activa, que es justamente lo que exige el criterio de sincronización. El crecimiento de la CPU es aproximadamente proporcional al trabajo realizado: de A a C, el throughput aumenta 21 veces y la CPU, 19. El entorno de réplica arrojó valores consistentes: CPU media de 0,19 %, 0,51 %, 1,98 % y 0,16 % y RSS de 16,0 a 17,7 MB en los escenarios A a D, con conservación en las 12 corridas.

TABLA V
RESULTADOS DE CPU, MEMORIA Y THROUGHPUT (MEDIA DE 3 CORRIDAS DE 20 S, ENTORNO PRINCIPAL)

Escenario	Hilos	CPU media (%)	CPU máx. (%)	RSS media (MB)	Paquetes/s	Cierre (s)	Conservación
A: 2 T / 3 O	9	0,17 ± 0,03	2,0	15,33	9,3	0,59	3/3
B: 10 T / 10 O	24	0,63 ± 0,04	4,0	15,37	40,7	0,58	3/3
C: 50 T / 50 O	104	3,20 ± 0,08	6,0	17,21	196,0	0,62	3/3
D: A sin supervisor	8	0,15 ± 0,04	2,0	15,25	10,8	0,50	3/3
B larga (120 s)*	24	0,51	4,0	16,34	36,7	0,52	1/1

Nota: T = camiones, O = operarios. La columna Cierre mide el tiempo desde SIGINT hasta la finalización del proceso. Conservación indica las corridas en que los paquetes entregados igualaron a los registrados. (*) Corrida hecha en el entorno de réplica.

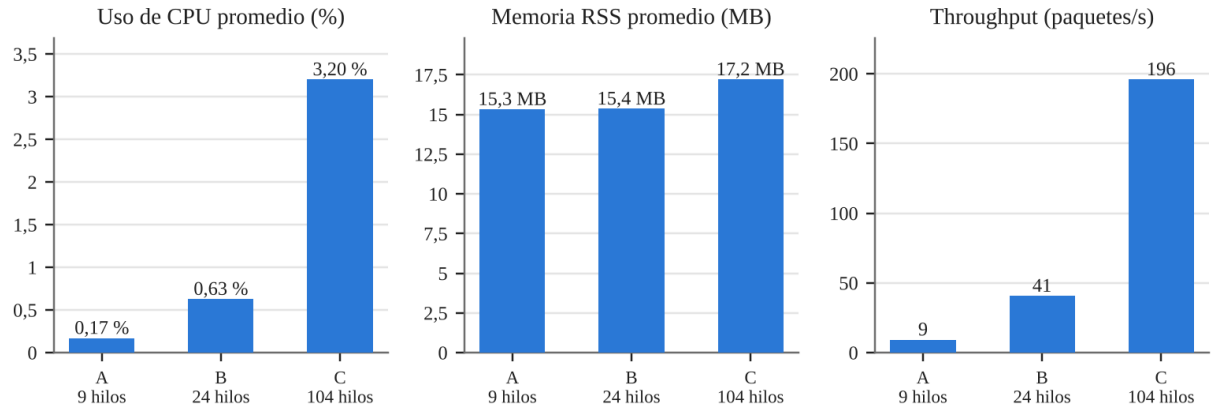


Fig. 2. Uso de CPU, memoria RSS y throughput por escenario. Promedio de 3 corridas de 20 s por escenario en el entorno principal (WSL2).

La memoria se mantuvo entre 15 y 18 MB. Pasar de 9 a 104 hilos agregó 1,88 MB, unos **20 KB por hilo** (16,8 KB en el entorno de réplica). Este costo marginal es bajo porque el RSS solo cuenta las páginas de la pila efectivamente usadas, no el espacio reservado. En la corrida de 120 s, el RSS aumentó apenas 0,09 MB mientras se procesaban 4403 paquetes, y el inventario guarda solo contadores por SKU. Por lo tanto, no hay evidencia de fugas de memoria.

El supervisor reduce el throughput de 10,8 a 9,3 paquetes por segundo (−14 %), un costo esperable por las pausas que impone. Agrega además 0,09 s al cierre, porque su espera también debe interrumpirse. En todos los escenarios el cierre tardó menos de 0,7 s, valor acotado por el *timeout* de 0,5 s de las operaciones sobre la cola. La escalabilidad del throughput está limitada por el diseño de la simulación (los camiones llegan cada 1 s y los operarios simulan 0,2 s por paquete) y no por el intérprete. Para cargas intensivas en CPU, el bloqueo global del intérprete (GIL) impediría aprovechar varios núcleos con hilos [18].

D. Análisis crítico del código generado

Como **fortalezas**, el código generado adoptó desde el inicio abstracciones de alto nivel que reducen la superficie de error [6]:

- objetos de dominio inmutables y una cola bloqueante como único canal entre productores y consumidores;
- lecturas por instantáneas inmutables;
- *timeouts* en toda operación bloqueante y una clase base que impide que un hilo falle en silencio, porque ante una excepción ordena el cierre global;
- la persistencia aplica el protocolo recomendado para escrituras resistentes a caídas (archivo temporal, `fsync` y renombrado atómico) [17];
- el lock no se mantiene durante la E/S de disco ni la de terminal, lo que evita tanto la protección parcial como la excesiva;
- el estilo es consistente (anotaciones de tipo, docstrings, constantes centralizadas).

Las **debilidades y errores** se descubrieron durante el desarrollo y constituyen el hallazgo central del trabajo:

- **Pausa por sondeo.** `threading.Event` no permite esperar a que la señal se desactive, por lo que el agente implementó la pausa consultando el estado cada 0,1 s. No consume CPU de forma apreciable, pero no es una espera puramente bloqueante; una `threading.Condition` o un evento invertido lo resolverían.
- **Portabilidad de la E/S.** La primera versión del exportador pasaba las pruebas en Linux, pero en Windows falló una prueba (18 lecturas exitosas sobre 20 requeridas), porque `os.replace()` lanza `PermissionError` si otro hilo tiene el archivo abierto. Sin corrección, ese error habría detenido toda la simulación. Se corrigió con reintentos acotados y omitiendo el ciclo fallido.
- **Bloqueo ante una segunda interrupción.** Un agente revisor detectó que un segundo Ctrl+C durante el cierre dejaba sin activar `producers_finished`. Los operarios, que no son hilos *daemon*, mantenían vivo el proceso indefinidamente. El defecto venía de las fases iniciales y lo detectó una revisión automática, no las pruebas. Se corrigió con un bloque `try/finally` y una prueba dedicada.
- **Requisitos en conflicto.** Para no ensuciar el panel se fijó el nivel de registro por defecto en WARNING, lo que ocultó el resumen final, que se emitía como registro INFO. Se corrigió imprimiendo el resumen por la salida estándar.
- **Efectos colaterales de las herramientas.** La configuración de gentle-ai creó más de 120 archivos de agentes dentro del repositorio y uno en una ruta errónea. Hubo que excluirlos manualmente del control de versiones. Además, sus revisiones automáticas compararon contra una base vacía ("26 archivos, 1951 líneas" para un cambio de 8 archivos), lo que inflaba el costo de cada revisión.

E. Tokens utilizados

La Tabla VI resume los tokens registrados en cada fase del proyecto. En las sesiones con Claude Code, casi todo el volumen corresponde al contexto que el agente vuelve a enviar en cada llamada (código del proyecto,

conversación, instrucciones y memoria), que en las fases 3 y 4 se sirvió desde la caché del proveedor a una fracción del precio normal. Los tokens de salida, en cambio, corresponden a lo que el modelo efectivamente generó: código, pruebas y mensajes de *commit*.

TABLA VI
CONSUMO DE TOKENS POR FASE

Fase	Herramienta / modelo	Tokens	Detalle
0 (planificación)	Gemini 3.1 Pro	≈ 6 700	Elección del problema, prompt inicial y plan de fases (estimado por extensión del texto)
0 a 2 (código inicial, panel, chaos)	Claude Code + gentle-ai / Claude Opus 5.5	509 600	4 sesiones: 282 900 de mensajes, 97 000 de memoria de gentle-ai y 129 800 de instrucciones, herramientas y skills
3 (persistencia)	Claude Code + gentle-ai / Claude Opus 5.5 y Haiku 5.5	2 921 206	Salida 18 806; caché 2 899 100; 25 llamadas; 322 líneas agregadas; US$ 1,74
4 (CLI, correcciones, README)	Claude Code + gentle-ai / Claude Opus 5.5 y Haiku 5.5	13 029 371	Salida 49 471; caché 12 973 400; 83 llamadas; 480 líneas agregadas; US$ 4,95

Nota: con un plan de suscripción el costo es teórico, equivalente al precio de la API.

En las fases 0 a 2, una parte importante del contexto no dependió del trabajo realizado. En cada sesión, entre 54 700 y 59 200 tokens ya estaban ocupados antes del primer mensaje: las instrucciones, las herramientas, las *skills* y, sobre todo, unos 24 000 tokens de archivos de memoria de gentle-ai. Esa carga fija representa entre el 36 % y el 58 % del contexto de cada sesión, y su peso relativo es mayor cuanto más corta es la sesión.

La comparación entre las fases 3 y 4 muestra un costo poco visible del desarrollo asistido. La fase 4 generó 2,6 veces más tokens de salida que la 3 y agregó 480 líneas frente a 322, pero leyó 4,5 veces más tokens desde la caché y requirió 83 llamadas al modelo contra 25. La diferencia se explica por las dos revisiones automáticas del proyecto completo y los ciclos de corrección posteriores. Cada una de esas llamadas reenvía el contexto entero, así que el costo crece con el tamaño del proyecto y con la cantidad de iteraciones, no con el tamaño del cambio. Entre ambas fases se procesaron unos 15,9 millones de tokens, con un costo equivalente de US$ 6,69.

F. Objetivos en curso y futuros

- Refactorizar las nueve funciones que exceden las 15 líneas para cumplir el estándar de la cátedra.
- Reemplazar el sondeo de la pausa de mantenimiento por una espera bloqueante con `threading.Condition`.
- Medir también de forma nativa en Windows (sin WSL) y, en futuros trabajos, registrar /cost en todas las sesiones para obtener una métrica de tokens homogénea.
- Comparar la misma consigna generada con distintos modelos, para separar el aporte del modelo del aporte del proceso.

V. CONCLUSIONES

El VibeCoding permitió construir en pocos días un programa concurrente no trivial, funcional y con una batería de pruebas mayor que el propio código de producción (1069 frente a 962 líneas). Las métricas confirman que el resultado es correcto y eficiente: ningún paquete se perdió ni se duplicó en 25 ejecuciones de hasta 104 hilos en dos entornos, el uso medio de CPU no superó el 3,2 %, la memoria se mantuvo estable entre 15 y 18 MB y el cierre tardó menos de 0,7 s.

Las limitaciones observadas coinciden con la literatura [5], [9], [10]. Los defectos más relevantes no estaban en la lógica central de sincronización, que la IA resolvió con patrones probados, sino en los bordes: la portabilidad entre sistemas operativos, las interrupciones durante el cierre, los requisitos que se contradicen y las reglas implícitas que el modelo no conocía. Las pruebas automatizadas detectaron algunos de esos defectos y un agente revisor detectó otro, pero ninguno se habría corregido sin un desarrollador que ejecutara el programa, interpretara los fallos y decidiera qué cambiar.

La principal ventaja de estas herramientas es la velocidad con que producen código idiomático y bien probado a partir de restricciones precisas. La principal limitación es que solo resuelven lo que el prompt especifica. Por eso, el diseño del prompt, a partir de las rúbricas, y la supervisión de cada fase mediante pruebas, ejecución real y medición resultaron tan determinantes como el modelo utilizado.

VI. REFERENCIAS Y BIBLIOGRAFÍA

A. Referencias bibliográficas

[1] E. W. Dijkstra, "Cooperating sequential processes," en *Programming Languages*, F. Genuys, Ed. New York, NY, USA: Academic Press, 1968, pp. 43–112.

[2] C. A. R. Hoare, "Monitors: An operating system structuring concept," *Commun. ACM*, vol. 17, no. 10, pp. 549–557, oct. 1974.

[3] P. J. Courtois, F. Heymans y D. L. Parnas, "Concurrent control with 'readers' and 'writers'," *Commun. ACM*, vol. 14, no. 10, pp. 667–668, oct. 1971.

[4] E. G. Coffman, M. Elphick y A. Shoshani, "System deadlocks," *ACM Comput. Surv.*, vol. 3, no. 2, pp. 67–78, jun. 1971.

[5] S. Lu, S. Park, E. Seo e Y. Zhou, "Learning from mistakes: A comprehensive study on real world concurrency bug characteristics," en *Proc. 13th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, Seattle, WA, USA, 2008, pp. 329–339.

[6] M. Herlihy y N. Shavit, *The Art of Multiprocessor Programming*. Burlington, MA, USA: Morgan Kaufmann (Elsevier), 2008.

[7] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li et al., "Large language models for software engineering: A systematic literature review," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 8, pp. 1–79, nov. 2024.

[8] A. Ziegler, E. Kalliamvakou, X. A. Li, A. Rice, D. Rifkin, S. Simister et al., "Measuring GitHub Copilot's impact on productivity," *Commun. ACM*, vol. 67, no. 3, pp. 54–63, mar. 2024.

[9] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt y R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," en *Proc. IEEE Symp. Security and Privacy (SP)*, San Francisco, CA, USA, 2022, pp. 754–768.

[10] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram y A. Bhatele, "Can large language models write parallel code?," en *Proc. 33rd Int. Symp. High-Performance Parallel and Distributed Computing (HPDC)*, Pisa, Italia, 2024, pp. 281–294.

[11] A. Karpathy. (2 feb. 2025). *There's a new kind of coding I call "vibe coding"* [en línea]. Consultado: 15 oct. 2026. Disponible: `https://x.com/karpathy/status/1886192184808149383`

[12] R. Sapkota, K. I. Roumeliotis y M. Karkee, "Vibe coding vs. agentic coding: Fundamentals and practical implications of agentic AI," arXiv:2505.19443, may. 2025.

[13] N. Fenton y J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3.a ed. Boca Raton, FL, USA: CRC Press, 2014.

[14] R. Jain, *The Art of Computer Systems Performance Analysis*. New York, NY, USA: Wiley, 1991.
[15] A. Georges, D. Buytaert y L. Eeckhout, "Statistically rigorous Java performance evaluation," en *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming, Systems, Languages and Applications (OOPSLA)*, Montreal, Canadá, 2007, pp. 57–76.
[16] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds et al., "Chaos engineering," *IEEE Softw.*, vol. 33, no. 3, pp. 35–41, may.–jun. 2016.
[17] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswany, A. C. Arpaci-Dusseau y R. H. Arpaci-Dusseau, "All file systems are not created equal: On the complexity of crafting crash-consistent applications," en *Proc. 11th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, Broomfield, CO, USA, 2014, pp. 433–448.
[18] Python Software Foundation. (2024). *threading — Thread-based parallelism*, Python 3.13 Documentation [en línea]. Consultado: 15 oct. 2026. Disponible: `https://docs.python.org/3.13/library/threading.html`

B. Bibliografía

G. R. Andrews, *Foundations of Multithreaded, Parallel, and Distributed Programming*. Reading, MA, USA: Addison-Wesley, 2000.
Gentleman Programming. *gentle-ai* [en línea]. Consultado: 12 oct. 2026. Disponible: `https://github.com/Gentleman-Programming/gentle-ai`
Google. *Google Python Style Guide* [en línea]. Consultado: 15 oct. 2026. Disponible: `https://google.github.io/styleguide/pyguide.html`
G. Rodola. *psutil documentation* [en línea]. Consultado: 15 oct. 2026. Disponible: `https://psutil.readthedocs.io`